Rota Food — roteiro da entrevista

Deck: abra apresentacao-entrevista.html no Chrome, `F` para tela cheia, setas para avançar, `N` para ver as notas no canto.

Vaga: analista de automação sênior
Duração alvo: 8–10 minutos + perguntas
Frase de uma linha: o Rota Food planeja o turno do entregador — captura o pedido, sugere a sequência e fecha o dia com km e custo, sem conta, sem servidor e sem substituir o Google Maps.

Números que você pode citar com segurança (estão no produto):

- Versão 1.1.2, Android, MIT, sem anúncios
- Cidade padrão: São Bernardo do Campo
- Nominatim: User-Agent `RotaFood/1.1`, cerca de 1 pedido por segundo, até 5 candidatos
- OCR: até 5 fotos de comanda, no celular
- Prioridade na rota: urgente 0,4 / alta 0,7 / normal 1,0
- Tempo estimado: km em linha reta ÷ 28 km/h
- Óleo: lembrete a cada 1000 km

Roteiro falado (timed)

Slide 1 — Abertura (~15 s)

Meu nome é [seu nome]. Trouxe um projeto que eu desenvolvi, o Rota Food. É um planejador de rotas para quem entrega comida — pizzaria, restaurante, dark kitchen. A ideia não era fazer um iFood. Era automatizar o turno de quem já tem a lista de pedidos na mão.

Slide 2 — O processo que eu vi (~1 min)

O processo real é simples e cansativo. O pedido chega no WhatsApp ou numa comanda impressa. A pessoa copia endereço um por um, decide a ordem de cabeça, abre o Google Maps e sai. No fim do dia, quilômetro e gasolina quase nunca viram número.

O que eu vi foi um fluxo de automação clássico: dado não estruturado na entrada, decisão no meio, e um sistema que já funciona na saída — o Maps. Faltava a camada que organiza o turno.

Slide 3 — Problema e para quem (~45 s)

O usuário é o entregador com várias paradas no mesmo turno, não o cliente pedindo pizza no aplicativo.

A dor é tempo e erro operacional: copiar endereço, achar o ponto, escolher a próxima entrega, e não saber se o dia pagou a gasolina. Eu me prestei a resolver isso no celular, de graça, sem pedir conta.

Slide 4 — Visão de produto (~1 min)

A visão é um app Android, gratuito, que cabe no bolso do motoboy. Offline-first: a lista fica no SQLite do aparelho. Sem servidor meu, sem login.

O produto também define o que ele não é. Ele não substitui o Google Maps. Distância no app é estimativa em linha reta. Rua, trânsito e bloqueio ficam com o Maps. Isso é limite de produto, não descuido.

Segurança no trânsito entra na visão: configura parado, navega com o Maps, não mexe no app em movimento.

Slide 5 — O fluxo automatizado (~1,5 min)

O pipeline é este.

Entrada: cola um bloco de WhatsApp ou fotografa até cinco comandas. O OCR roda no aparelho; o texto da comanda não sobe para a nuvem. Um parser tenta nome, telefone, valor e endereço.

Localização: consulto o Nominatim, até cinco candidatos. A pessoa confirma no mapa ou arrasta o pino. Só depois disso o ponto entra no cache. Sem internet, dá para ver, editar e concluir o que já está salvo.

Decisão: vizinho mais próximo a partir da pizzaria ou do GPS, com peso de prioridade. Entrega sem coordenada vai para o fim.

Saída: Navegar abre o Google Maps com o **texto do endereço**, não com a lat/lng do OSM — porque o Maps costuma achar melhor o endereço escrito. Marca entregue, recalcula o restante. Inicia o turno no GPS, vê km, gasolina estimada, óleo a cada mil quilômetros, e manda o relatório no WhatsApp.

Isso, para mim, é automação: ingestão, normalização, decisão, handoff e fechamento.

Slide 6 — Onde automatizar e onde não (~45 s)

Eu automatizo o repetitivo. O humano fica no ponto em que o erro custa caro: o pino no mapa errado manda o motoboy para o lugar errado.

E eu componho sistemas em vez de reconstruir. O Maps já navega. O WhatsApp já é o canal do relatório. O app orquestra; não tenta ser GPS de rua.

Slide 7 — Tecnologias (~1,5 min)

É Flutter, Dart, APK Android. Estado com Provider. Dados no SQLite. Sem backend próprio.

Integrações: HTTP no Nominatim, mapa com flutter_map, GPS com geolocator, intent com url_launcher para Maps, WhatsApp e discador. OCR com Google ML Kit no dispositivo.

A rota é Haversine mais vizinho mais próximo. Não é um TSP ótimo. É uma heurística barata, previsível, suficiente para um turno de poucas dezenas de paradas, e fácil de explicar para quem usa.

Slide 8 — Por que essas escolhas (~1 min)

Nominatim em vez de Google Geocoding: custo zero no MVP, sem chave. OCR no aparelho: privacidade e zero fatura de API. Handoff para o Maps: não reinventar o que já está no processo.

Se eu tivesse orçamento de API e volume, o próximo passo natural seria distância de rua com OSRM ou Directions. Eu não coloquei isso no MVP porque a estimativa honesta já serve para planejar, e a navegação de verdade já existe.

Slide 9 — O que já está no aparelho (~45 s)

Isso não é só ideia. Está no APK: lote, OCR, confirmação, rota, turno com GPS, relatório e histórico local. Versão 1.1.2.

Slide 10 — Fechamento (~30 s)

O que eu faria depois, com critério: ruas reais quando a linha reta deixar de ser boa o bastante; parser e OCR mais tolerantes; nuvem só se o usuário pedir para sincronizar dois aparelhos — não por padrão.

Posso mostrar o fluxo no celular ou ir nas escolhas de automação. Obrigado.

Demo de 60 segundos (plano B)

Celular já com o APK, estabelecimento cadastrado, GPS ok, rede ok. Ensaie uma vez.

1. **0–10 s** — Home: “esta é a lista do turno”.
2. **10–25 s** — Colar endereços (ou OCR se a câmera estiver rápida): dois ou três endereços de SBC, não dez.
3. **25–40 s** — Confirmar o pino no mapa: “o humano valida; o app não confia cego no OSM”.
4. **40–50 s** — Organizar rota: “vizinho mais próximo, prioridade entra no peso”.
5. **50–60 s** — Navegar: mostra que abre o Maps com o endereço escrito. Se der tempo: iniciar turno / relatório.

Se a geocodificação falhar: arraste o pino e siga. Não peça desculpa longa.

Não mexa no app “andando”. Diga isso em voz alta se alguém sugerir simular no corredor.

FAQ (10 perguntas)

1. Por que vizinho mais próximo e não TSP / Google Directions?
Porque o turno é curto, a heurística é instantânea e explicável, e a navegação de rua já é do Maps. TSP exato cresce rápido e não muda a vida de quem tem 8–15 paradas. Directions tem custo e chave; eu deixei para quando a estimativa de km deixar de ser honesta o bastante.

2. A linha reta não mente o tempo?
Mente um pouco, e o produto fala isso. Serve para ordenar e dar uma ordem de grandeza. Quem guia no asfalto é o Maps.

3. O Nominatim erra o ponto. E aí?
Cinco candidatos, confirmação no mapa, arrastar o marcador. Cache só depois de confirmar. Navegar usa o texto do endereço, não a coordenada do OSM.

4. Cliente e telefone vão para a nuvem?
Não. SQLite local. OCR local. O que sai do aparelho é o texto do endereço no Nominatim (serviço público) e o que a pessoa manda no WhatsApp ou abre no Maps.

5. Como você mediria sucesso / ROI dessa automação?
Tempo para montar o turno (do primeiro endereço até o Navegar), retrabalho de endereço errado, km do GPS versus “no olho”, e se o relatório do dia passou a ser enviado. Em um piloto: 10 turnos com o app vs 10 sem, mesmos entregadores.

6. O que você faria com orçamento de API?
Geocoding mais estável, matriz de distância em rua (OSRM self-host ou Directions), talvez

OCR na nuvem só como fallback. Eu ainda manteria confirmação humana no pino.

7. Por que Flutter e não nativo / web?
Um APK, UI de alvo grande, acesso a câmera, GPS e intent. Web não instala no bolso do motoboy do mesmo jeito. Nativo duplicaria esforço num projeto solo.

8. Sem backend não é limitação técnica?
É decisão. Sem conta, sem fatura, sem LGPD de servidor meu, funciona no fundo do restaurante se a lista já estiver no celular. O preço é não sincronizar dois aparelhos. Isso é V3 opcional, não o MVP.

9. Como isso se parece com o trabalho de analista de automação?
Mapear AS-IS, achar dado sujo na entrada, normalizar, colocar regra de decisão explícita, integrar o que já existe, deixar humano no risco, medir no fechamento do turno. O app é o robô desse processo.

10. Se o restaurante quisesse isso para a cozinha inteira?
Aí muda o produto: vários usuários, fila, papel da pizzaria vs do entregador, talvez sync. Eu não fingiria que o APK solo resolve operação de frota. Escopo atual é o turno de uma pessoa no próprio celular.

Como usar na segunda

1. Abrir o HTML no Chrome, `F`, falar o roteiro; o slide só apoia.
2. Celular com APK no bolso.
3. Se pedirem código: camadas em `lib/` — `services/route_optimizer.dart`, `core/utils/address_parser.dart`, `services/ocr_service.dart`, `docs/architecture.md`.
4. Não improvise número que não está nesta página.